VAKRATRON SYSTEMS

WHITEPAPER · AI INFRASTRUCTURE AND GPU CLUSTERS

GPU Infrastructure for Enterprise AI

Sizing, building and running GPU clusters for training and inference, from memory maths to power, fabric, scheduling and cost per GPU-hour

Written for
CIOs, CTOs, heads of AI and data, infrastructure and data centre architects

Version 1.0, October 2026
vakratronsys.com

Contents

01 Executive summary

02 Why GPU projects disappoint

03 Training and inference sizing, done properly

04 GPU and deployment options compared

05 A decision framework

06 Reference architecture

07 Sizing, power and cost

08 Security, sharing and governance

09 Running it: scheduling, utilisation and failures

10 A practical roadmap

11 Ten common mistakes

12 GPU readiness checklist (36 points)

13 Glossary

14 About Vakratron Systems

What you will take away

- How to size GPU memory for training and inference, including the KV cache that most estimates miss
- H100, H200, L40S and newer GPUs compared on what each is actually good for
- Rail-optimised InfiniBand or RoCE fabric, and how many switches a cluster needs
- Storage tiers and checkpoint maths, so GPUs are not left waiting on data
- Power and cooling per rack, and when liquid cooling becomes necessary
- Cost per GPU-hour with a worked example, and the utilisation at which buying beats renting
- A 36-point checklist covering workload, facility, fabric, platform and operations

01

Executive summary

GPUs are the most expensive part of an AI platform and the easiest to waste. They sit idle when the network between servers is too slow, when storage cannot feed them, when the data centre cannot power and cool them, or when nobody can see who is using them. A GPU cluster is a system: servers, fabric, storage, scheduler, power and cooling, all designed around the GPUs.

This paper sets out how to size, build and run that system for training and inference, with the formulas and planning figures needed to check a vendor proposal or build a business case.

Five things to take away

1. **Start from the workload, not the GPU.** Which models, training or inference, how many concurrent users, what context length and what latency. These decide the GPU, the count and the fabric.
2. **Memory decides the GPU count.** Model weights are only part of it. For inference, the KV cache for many users with long prompts can be as large as the model. For training, optimiser state needs about 16 bytes per parameter.
3. **Fabric, storage and power are not extras.** Multi-server training needs a dedicated 400 Gb/s per GPU fabric. One 8-GPU server draws about 10 kW, more than many enterprise racks are built for. Check the facility before ordering.
4. **Utilisation decides the cost.** An owned cluster at 70 percent utilisation costs roughly ₹150 to 250 per GPU-hour over five years. At 40 percent it costs as much as renting, or more.
5. **Plan for failure.** In long runs something fails. Frequent checkpoints, burn-in testing, health monitoring and a spare server turn a failure into minutes of lost work, not days.

02

Why GPU projects disappoint

The pattern is familiar. The servers are ordered quickly because GPUs are scarce, and the rest of the system is designed afterwards. Months later the cluster is installed but slow, half-used or not running at all.

What we find	What happens
The data centre is not ready	Racks are built for 5 to 10 kW. Each 8-GPU server needs about 10 kW. The servers arrive and there is nowhere to plug them in.
GPUs wait on the network	Training across servers runs over ordinary data centre Ethernet. 32 GPUs deliver the speed of 12.
Storage cannot keep up	Data loads and checkpoints go to a general-purpose filer. GPUs sit idle for minutes at every checkpoint.
Wrong GPU for the job	Training-class GPUs serve a small model, or inference GPUs are asked to train across servers.
Nobody can see utilisation	Teams queue for access while allocated GPUs run at 20 percent. No one can say who used what.
No plan for failure	A single GPU fault stops a week-long job, and the last checkpoint is a day old.

> **In short:** GPU projects fail at the edges of the GPU: power, fabric, storage and scheduling. Design those first.

03

Training and inference sizing, done properly

Memory per parameter

A model's size in memory is its parameter count times the bytes used per parameter. That depends on precision and on whether you are training or serving.

Use	Bytes per parameter	70-billion-parameter model
Inference at 16-bit (BF16 or FP16)	2	About 140 GB
Inference at 8-bit (FP8 or INT8)	1	About 70 GB
Inference at 4-bit	About 0.5	About 35 GB, test quality on your own tasks
Full training, mixed precision with Adam	About 16 (weights, gradients, optimiser state)	About 1.1 TB, plus activations
Parameter-efficient fine-tuning (LoRA)	Base weights plus a small adapter	Close to inference memory

The KV cache

When a model generates text, it keeps a cache of keys and values for every token in the conversation so it does not recompute them. That cache grows with context length and with the number of users served at once. For a 70B model with 8,000-token contexts, each active user needs about 2.7 GB. Thirty-two users need more memory for the cache than for an 8-bit copy of the model. Most undersized inference platforms got this wrong.

Throughput and latency

Inference has two phases. Reading the prompt (prefill) is compute-heavy. Generating each new token (decode) is limited by memory bandwidth, because the weights are read once per step. Model servers such as vLLM batch many users into each step, so one read of the weights serves dozens of requests. This is why memory bandwidth (3.35 TB/s on H100, 4.8 TB/s on H200) matters as much as compute for serving. Measure time to first token and tokens per second per user, at your real concurrency.

How training spreads across GPUs

Training splits work across GPUs in three ways: data parallel (each GPU sees different data), tensor and pipeline parallel (the model itself is split), and sharded optimiser state (FSDP or DeepSpeed ZeRO). All of them exchange results after every step. Inside a server, NVLink carries this traffic at very high speed; between servers, the compute fabric does. This is why the fabric decides whether 32 GPUs behave like 32 or like 12.

04

GPU and deployment options compared

GPU choice follows the workload. Most enterprise platforms need two classes: training-class GPUs for fine-tuning and large models, and lower-cost GPUs for serving small and mid-size models.

GPU	Memory and bandwidth	Power	Good for	Not designed for
H100 SXM	80 GB HBM3, 3.35 TB/s	Up to 700 W	Training and fine-tuning, serving large models	Cost-sensitive serving of small models
H200 SXM	141 GB HBM3e, 4.8 TB/s	Up to 700 W	Same as H100, with more room for large models and long contexts	Small workloads that do not need the memory
Newer generation (B200 class)	About 180 GB HBM3e, about 8 TB/s	About 1 kW	Large-scale training and serving at high density	Facilities without liquid cooling or high rack power
L40S (PCIe)	48 GB GDDR6, 864 GB/s	350 W	Serving models up to about 30B, vision, graphics	Multi-server training
L4 (PCIe)	24 GB GDDR6, 300 GB/s	72 W	Small models, video analytics, edge	Large models or training

As planning ranges in 2026, an 8-GPU H100 or H200 class server costs roughly ₹2 to 3 crore, and a PCIe server with four to eight L40S-class GPUs roughly ₹40 lakh to ₹1 crore. Prices and lead times move with supply; treat these as budget ranges, not quotations.

Buy, host or rent

Option	Good for	Trade-off
Buy and host in your own data centre	Steady, long-term use; data that must stay on your premises	Large upfront cost; your facility must handle the power density
Buy and place in colocation	The same, when your own data centre cannot take high-density racks	Colocation contract and remote hands; still a capital purchase
Rent from a GPU cloud	Experiments, short projects, bursts of training	Highest cost per GPU-hour over time; capacity can be tight

> **A common pattern:** rent for three to six months to learn real usage, then buy for the steady base and keep renting for peaks.

05

A decision framework

Five questions, answered in order, settle most GPU decisions:

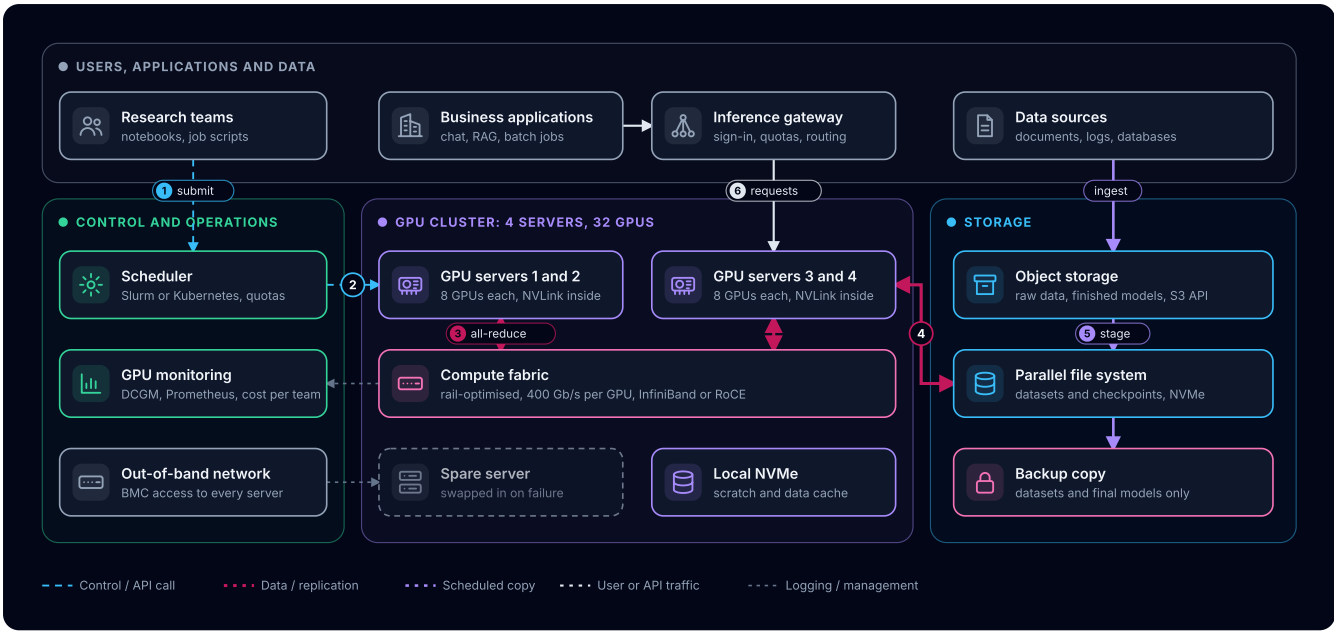
1. **Training, inference or both?** Inference-only platforms rarely need a high-speed fabric. Training across servers always does.
2. **How large is the model, and at what precision?** This sets the memory floor per copy of the model, and so the GPU type.
3. **How many users, with what context and latency?** This sets the KV cache and the number of model copies. A chat assistant needs a fast first token; an overnight document job does not.
4. **How steady is demand?** Above roughly 50 percent sustained utilisation, owning usually costs less than renting. Below it, rent.
5. **Where will it run?** Power per rack, cooling, floor loading and data residency decide between your data centre, colocation and a GPU cloud.

Workload	Typical starting point
Internal assistant on a 7B to 8B model, a few hundred users	Two to four L40S-class GPUs, or MIG slices of one H100; standard Ethernet
Enterprise assistant on a 70B model at 8-bit, 30 to 50 concurrent users	Four H100 or two H200 per copy, two copies for resilience; standard Ethernet
Fine-tuning 8B to 70B models on company data	One to four 8-GPU servers with a 400 Gb/s per GPU fabric
Shared research cluster for several teams	4 to 32 servers, rail-optimised fabric, Slurm with quotas, parallel file system

06

Reference architecture

The diagram shows a 32-GPU cluster used for both fine-tuning and serving. Compute, storage and management traffic each have their own network, and the scheduler sees every GPU.



Numbered flows: (1) teams submit jobs to the scheduler under their quota; (2) the scheduler places each job on whole servers, close together on the fabric; (3) during training, GPUs exchange gradients after every step over the compute fabric; (4) datasets are read from, and checkpoints written to, the parallel file system over a separate storage network; (5) raw data and finished models live in object storage and are staged to the fast tier when needed; (6) applications reach models through an inference gateway that signs in each caller, applies quotas and routes requests. GPU and fabric health stream to monitoring, and every server can be reached out of band.

The rail-optimised fabric

Each GPU has its own 400 Gb/s network port, so an 8-GPU server has eight fabric ports. In a rail-optimised design, GPU 1 of every server connects to leaf switch 1, GPU 2 to leaf 2, and so on. Traffic between same-numbered GPUs crosses one switch, which keeps latency low and predictable. Every leaf connects to every spine, with as much bandwidth up as down, so the fabric is non-blocking.

Cluster size	Fabric ports	Typical switches (64-port 400G)
4 servers, 32 GPUs	32 × 400G	One switch, with room to grow
16 servers, 128 GPUs	128 × 400G	Rail leaves plus a small spine layer
32 servers, 256 GPUs	256 × 400G	8 rail leaves and 4 spines: 256 ports down, 256 up

InfiniBand or RoCE Ethernet

Both work. InfiniBand (NDR 400G) is lossless by design, proven at scale for training and simpler to tune, but brings separate skills and mostly one vendor. RoCE runs RDMA over standard 400G Ethernet with priority flow control and congestion notification, is familiar to network teams and offers wider vendor choice, but needs careful configuration and testing to match InfiniBand under load. Choose on your team’s skills and existing network more than on benchmarks. Keep storage and management traffic off the compute fabric either way.

07

Sizing, power and cost

Inference memory

```
Memory = parameters × bytes per parameter + KV cache + about 10% overhead
KV cache per token = 2 × layers × KV heads × head dimension × bytes per value
```

For a 70B model with 80 layers, 8 KV heads of dimension 128 and a 16-bit cache, one token needs 2 × 80 × 8 × 128 × 2 = 327,680 bytes, about 0.33 MB.

Item	Arithmetic	Result
Weights at 8-bit	70 billion × 1 byte	70 GB
KV cache per user, 8,000-token context	0.33 MB × 8,192 tokens	About 2.7 GB
KV cache for 32 concurrent users	2.7 GB × 32	About 86 GB
Total with 10% overhead	(70 + 86) × 1.1	About 172 GB
Fits on	172 GB against 80 GB or 141 GB per GPU	4 x H100 with headroom, or 2 x H200

Training time

```
GPU-hours ≈ 6 × parameters × training tokens ÷ (GPU peak FLOPS × utilisation × 3,600)
```

Continued training of an 8B model on 100 billion tokens needs $6 \times 8{\times}10^{9} \times 10^{11} = 4.8{\times}10^{21}$ operations. An H100 delivers about 990 TFLOPS at BF16; at a realistic 40 percent utilisation that is about 400 TFLOPS, so the job needs about 3,300 GPU-hours: roughly four and a half days on 32 GPUs.

Checkpoints

Full training state is about 16 bytes per parameter: about 130 GB for an 8B model and 1.1 TB for 70B. At 10 GB/s, the 70B checkpoint takes about two minutes to write; at 50 GB/s, about 22 seconds. How often to checkpoint depends on how often jobs are interrupted:

```
Checkpoint interval ≈ √(2 × checkpoint write time × mean time between interruptions)
```

With a two-minute write and one interruption a week (10,080 minutes), the interval is √(2 × 2 × 10,080), about 200 minutes: checkpoint every three hours or so. Asynchronous checkpointing, where the framework supports it, hides most of the write time.

Power and cooling per rack

System or option	Planning figure
8-GPU H100 or H200 server	Up to about 10.2 kW
8-GPU B200-class server	Up to about 14.3 kW
Rack-scale liquid-cooled systems (GB200 NVL72 class)	Around 120 kW per rack
Air cooling with aisle containment	Practical up to about 20 to 25 kW per rack
Rear-door heat exchangers	Roughly 30 to 50 kW per rack
Direct liquid cooling to the chips	50 to 100 kW per rack and more

Four H100 servers draw about 4 × 10.2 = 41 kW; with switches, storage and management servers, about 47 kW. In a room built for 8 kW racks that means six mostly empty racks and hot spots. With rear-door heat exchangers at 25 kW per rack it fits in two. Plan with the vendor's maximum figures: under training load, real draw is close to the maximum.

Cost per GPU-hour

```
Cost per GPU-hour = five-year cost ÷ (GPUs × 43,800 hours × utilisation)
```

Five-year cost, 32-GPU cluster (planning ranges)	₹ crore
4 × 8-GPU H100 or H200 class servers	8 to 12
Compute fabric: switch, cables and optics	0.5 to 1
Parallel file system and object storage	1 to 2.5
Management servers and Ethernet	0.3 to 0.5
Power, cooling and space (about 57 kW at the meter)	2 to 3.5
Support, maintenance and software	1 to 2
People: about two engineers	2 to 3
Total	**14.8 to 24.5**

32 GPUs provide 32 × 43,800 = 1.4 million GPU-hours in five years. At 70 percent utilisation that is about 980,000 used hours, so the cost is roughly ₹150 to 250 per GPU-hour. At 40 percent it rises to roughly ₹265 to 440, which is within or above the range for renting H100-class capacity on commitment. Utilisation, not the purchase price, decides whether owning pays.

08

Security, sharing and governance

Model weights, fine-tuning datasets and prompts are among the most sensitive data an organisation holds. A GPU cluster shared by several teams needs the same controls as any multi-tenant platform, plus a few of its own.

Sharing GPUs safely

Method	Isolation	Use it for
Whole servers per job or tenant	Strongest: separate hardware, separate fabric partitions	Training, regulated data, external tenants
MIG (H100, H200)	Hardware-partitioned memory and compute, up to 7 slices per GPU	Small models and notebooks that do not need a whole GPU
vGPU	Hypervisor-enforced, with licensing	GPU-backed virtual machines and desktops
Time-slicing	None between workloads: shared memory	Development within one trusted team only

Controls that matter

- **One identity.** Sign-in through your directory for the scheduler, notebooks and inference gateway, with roles and quotas per team.
- **Separate networks.** Fabric partitions (InfiniBand partition keys or VLANs on RoCE) between tenants, and management networks reachable only by administrators.
- **Data protection.** Encryption at rest on the file system and object store; datasets classified before they are used for training; personal data handled under the DPDP Act.
- **A model registry.** Only approved, versioned models are served, with a record of the data each was trained on.
- **Audit.** Who ran which job on which data, and who called which model, logged centrally. CERT-In expects incidents reported within six hours and logs kept for 180 days within India.

> **Air-gapped clusters:** for regulated or defence workloads the whole platform can run without internet access. Plan an internal mirror for containers, drivers, models and packages from the start; it is the part most often forgotten.

09

Running it: scheduling, utilisation and failures

Slurm or Kubernetes

	Slurm	Kubernetes with NVIDIA GPU Operator
Best at	Batch training, multi-node jobs, fair-share queues	Serving, mixed workloads, autoscaling, platform teams
Users who like it	Researchers used to HPC job scripts	Engineers used to containers and CI pipelines
Gang scheduling	Built in	Needs an add-on such as Kueue or Volcano

Many clusters run both: Slurm for training, Kubernetes for inference, with nodes moved between them as demand changes. Inference needs steady capacity during working hours; training can use what is left, especially at night, if the scheduler supports priorities and pre-emption.

Measuring utilisation

“Allocated” is not “busy”. Track both: the share of GPUs assigned to jobs, and how hard those GPUs actually work (from DCGM metrics). A well-run training cluster keeps 70 to 85 percent of GPUs allocated, with allocated GPUs genuinely busy. Publish a monthly report of GPU-hours and cost per team; it is the fastest way to find idle reservations.

Burn-in before handover

- Run DCGM diagnostics and stress tests on every GPU for 48 to 72 hours.
- Run NCCL all-reduce tests across all servers and compare bandwidth with the design figure. One bad cable or optic can halve it.
- Test the storage with real checkpoint sizes, and record the baselines for later comparison.

Handling failures

Because every GPU in a training job depends on every other at each step, one failure stops the whole job. At 256 GPUs, plan for an interruption roughly every week; at 32, perhaps once or twice a month. The response should be routine:

- Monitoring flags GPU errors (Xid events, memory errors), NVLink errors and fabric link flaps.
- The scheduler drains the faulty server, and jobs restart automatically from the last checkpoint.
- The spare server joins the pool; the faulty one is repaired under the vendor contract and re-tested before returning.
- Drivers, firmware and fabric software are kept at the same version on every server, upgraded in planned windows.

10

A practical roadmap

Power and cooling upgrades and hardware lead times usually set the pace. Start the facility check before anything else.

Stage	Typical duration	Outcome
1. Define the workload	2 to 3 weeks	Models, training or inference, users, context, latency targets, data sensitivity.
2. Facility check and design	3 to 4 weeks	Power and cooling per rack, sizing, fabric and storage design, bill of materials, cost per GPU-hour model.
3. Procure and prepare	6 to 16 weeks	Servers, switches and storage ordered; rack power, cooling and cabling upgraded in parallel.
4. Install and burn in	3 to 4 weeks	Racking, cabling, fabric validation, stress and NCCL tests, scheduler and monitoring set up.
5. Pilot	4 weeks	Two or three teams onboarded; quotas, checkpoints and failure runbook tested for real.
6. Production	Ongoing	All teams onboarded, monthly utilisation and cost report, capacity reviewed every quarter.

Questions to put to every vendor

Proposals are easier to compare when every bidder answers the same questions in writing:

- Which exact GPU, memory and NVLink configuration, and what is the maximum power per server under full load?
- Is the fabric rail-optimised and non-blocking? Show the switch, cable and optic count.
- What sustained write throughput does the storage reach with our checkpoint size, measured rather than quoted?
- Which burn-in and acceptance tests are included, with what pass criteria for NCCL bandwidth and DCGM diagnostics?
- How quickly is a failed GPU or server replaced, and are spares held in India?
- Which driver, firmware and fabric software versions are supported together, and for how long?

11

Ten common mistakes

1. **Ordering GPUs before checking the facility.** Power and cooling take longer to fix than servers take to arrive.
2. **Sizing inference from model size alone.** The KV cache for real users and contexts can double the memory.
3. **Training across servers on ordinary Ethernet.** The GPUs spend most of their time waiting.
4. **Putting storage traffic on the compute fabric.** Training slows in ways that are hard to diagnose.
5. **Using training-class GPUs to serve small models.** Or the reverse. Match the GPU to the job.
6. **Checkpointing too rarely.** A failure then costs a day of work on every GPU.
7. **Skipping burn-in.** A weak cable found in production costs far more than one found in week one.
8. **No quotas or per-team reporting.** GPUs sit reserved and idle while other teams queue.
9. **Time-slicing GPUs between untrusted tenants.** Memory is shared; use MIG or whole servers.
10. **Buying for peak demand.** Own the steady base and rent the peaks.

12

GPU readiness checklist (36 points)

Use this list to score your current position. Anything you cannot tick with evidence is a gap worth closing.

Workload and sizing

- [ ] Models, precision and training or inference needs written down for each use case
- [ ] Concurrent users, context length and latency targets agreed per use case
- [ ] GPU memory sized including KV cache and 10 percent overhead
- [ ] Training GPU-hours estimated from parameters, tokens and realistic utilisation
- [ ] Expected utilisation estimated, and buy, host or rent decided on that basis
- [ ] Growth for the next two to three years included

Facility

- [ ] Power per rack confirmed against vendor maximum figures, on A and B feeds
- [ ] Room-level spare power and cooling confirmed, not just per rack
- [ ] Cooling method chosen: air, rear-door or direct liquid
- [ ] Floor loading checked for fully loaded GPU racks
- [ ] Chilled water available where rear-door or liquid cooling is planned
- [ ] Facility upgrade lead time known and on the plan

Fabric and storage

- [ ] Dedicated compute fabric at 400 Gb/s per GPU for multi-server training
- [ ] Rail-optimised, non-blocking design with switch and cable counts confirmed
- [ ] Separate storage, in-band and out-of-band management networks
- [ ] Parallel file system sized for checkpoint write time, not just capacity
- [ ] Object storage for raw data and finished models, with staging to the fast tier
- [ ] Backup for datasets and final models

Platform and scheduling

- [ ] Scheduler chosen to match how teams work (Slurm, Kubernetes or both)
- [ ] Quotas, priorities and pre-emption configured per team
- [ ] GPU sharing policy: MIG or whole servers for untrusted tenants, no time-slicing
- [ ] Checkpoint interval set from write time and expected interruption rate
- [ ] Automatic job restart from the last checkpoint tested
- [ ] Spare server or vendor replacement agreement in place

Security and governance

- [ ] Single sign-on with roles for scheduler, notebooks and inference gateway
- [ ] Fabric and network partitions between tenants
- [ ] Encryption at rest for file system and object storage
- [ ] Training datasets classified, with personal data handled under the DPDP Act
- [ ] Model registry with approved, versioned models only
- [ ] Central audit logs kept for at least 180 days within India

Operations and cost

- [ ] 48 to 72 hour burn-in and NCCL tests passed on every server before handover
- [ ] DCGM monitoring of GPU errors, temperature and real utilisation
- [ ] Drivers and firmware kept at the same version on every server
- [ ] Monthly report of GPU-hours and cost per team
- [ ] Cost per GPU-hour tracked against the business case
- [ ] Failure runbook written and rehearsed

13

Glossary

Term	Meaning
HBM	High-bandwidth memory stacked next to the GPU chip; sets how much model fits and how fast it is read.
NVLink	NVIDIA’s high-speed link between GPUs inside one server.
KV cache	Keys and values kept for every token in a conversation so they are not recomputed.
Prefill and decode	Reading the prompt, then generating output tokens one step at a time.
All-reduce	The step where every GPU combines its results with every other GPU.
Rail-optimised fabric	Same-numbered GPUs on every server share a leaf switch, so most traffic crosses one switch.
RoCE	RDMA over Converged Ethernet: direct memory transfers over tuned, lossless Ethernet.
MIG	Multi-Instance GPU: hardware partitioning of one GPU into up to seven isolated slices.
DCGM	NVIDIA Data Center GPU Manager: health, diagnostics and utilisation metrics.
NCCL	NVIDIA’s library for communication between GPUs, also used to test fabric bandwidth.

About Vakratron Systems

Vakratron Systems LLP is a vendor-neutral infrastructure design firm, recognised as a startup by DPIIT. We design data centre, disaster recovery, cloud, GPU and AI platforms for enterprises and government buyers. Every design starts from the workload, compares realistic options with their costs, writes its assumptions down, and is tested before handover.

How we can help with GPU infrastructure

- **GPU sizing review:** from your models, users and latency targets, a written sizing of GPUs, servers, fabric, storage, power and cooling, with the assumptions shown.
- **Cluster design:** vendor-neutral reference design, bill of materials with alternatives, data centre readiness check and a cost per GPU-hour model.
- **Build assurance:** burn-in and fabric acceptance tests, scheduler and monitoring set-up, and runbooks for failures, so the cluster is busy from the first week.

Talk to us
connect@vakratronsys.com

Online
vakratronsys.com/advisory

This paper gives general guidance based on common enterprise requirements. Figures are planning ranges, not quotations. Apply them to your environment with care, or ask us to help. © 2026 Vakratron Systems LLP.